

DAYS 4-5 · FastAPI learning notes

Response Models, Status Codes, Errors & Body Updates

Day 4: response models · status codes · path operation configuration | Day 5: handling errors · body updates

Learning objectives

After working through these notes you should be able to:

- ✓ Use a return type or `response_model` to validate, filter and document what an endpoint returns.
- ✓ Keep sensitive fields (a password) out of responses with separate input/output models.
- ✓ Choose and set HTTP status codes, and configure how an endpoint appears in `/docs`.
- ✓ Raise `HTTPException` and write a custom exception handler.
- ✓ Explain PUT vs PATCH and do a partial update with `exclude_unset` + `model_copy`.

Note

This PDF combines the two original summaries (`day4_summary_clear_code(1).pdf` and `day5_summary_clear_code(1).pdf`) with a walkthrough of `day4-5/main.py`. All of their content is kept below.

1. Day 4 - Response model via the return type

Annotating a path operation function's return type (for example `-> Item`) makes FastAPI:

- **validate** the returned data,
- **filter** it down to the declared fields,
- **serialize** it via Pydantic, and
- add it to the **OpenAPI schema / docs**.

The `response_model` parameter

Use `response_model` on the decorator when the function returns something different (a dict, a database object) but you still want FastAPI to treat the result as a Pydantic model for docs, validation and filtering.

```
@app.post("/items/", response_model=Item)
async def create_item(item: Item) -> Any:
    return item
```

Remember

If both a return type and `response_model` are declared, **`response_model` takes priority**.

Filtering out sensitive fields

Declare a `UserIn` model with the password for input and a separate `UserOut` model without it for the response, so the password never leaks.

The inheritance trick

Define a model with the public fields, then a second model that inherits from it and adds the password. Annotate the function with the public model while returning the full one – FastAPI filters the response down to the public fields. This is exactly what your code does:

day4-5/main.py (shortened)

```
class UserIn(UserOut):
    password: str

@app.post("/user/", status_code=status.HTTP_200_OK, tags=["user"],
          summary="create a user", description="...",
          response_description="The created user")
async def create_user(user: UserIn) -> UserOut:
    return user
```

Verified: `POST /user/` with username, password and email returns only `{"username": ..., "email": ..., "full_name": null}` – the password is dropped.

In your code

Your comment on the commented-out version says “the password will be shown in the output” for `@app.post("/user/", response_model=UserOut) + -> Any`. In fact `response_model=UserOut` wins and **filters the password out**. The password leaks only when there is no response model at all, or when the response model is `UserIn`.

The file also defines `UserIn` and `UserOut` twice. Python just rebinds the names, and because both `UserOut` versions are identical it still works – but keep one definition per model in real code.

Returning a Response directly

If the return type is `Response` or a subclass (`RedirectResponse`, `JSONResponse`), FastAPI skips response model generation automatically. A union such as `Response | dict` fails; use `response_model=None` to disable response model generation.

Excluding unset, default or None values

`response_model_exclude_unset=True` includes only fields explicitly set on the object, omitting values that only came from defaults. Related options: `response_model_exclude_defaults=True` and `response_model_exclude_none=True`.

`response_model_include` / `response_model_exclude` are shortcuts for including or excluding named fields. Separate model classes are preferred, because with the shortcuts OpenAPI still shows the full model schema.

2. Day 4 – Response status codes

Set the HTTP status code with the `status_code` parameter on the decorator.

```
@app.post("/items/", status_code=201)
async def create_item(name: str):
    return {"name": name}
```

Range	Meaning	Examples / notes
100-199	Informational	rarely used directly; no response body allowed
200-299	Success	200 OK, 201 Created, 204 No Content
300-399	Redirection	may or may not have a body; 304 must not
400-499	Client error	404 Not Found, 401 Unauthorized, 422 validation error

Range	Meaning	Examples / notes
500-599	Server error	usually returned automatically for internal failures

Remember

Prefer constants such as `status.HTTP_201_CREATED` instead of memorizing numbers. Your `create_user` uses `status.HTTP_200_OK`; for a create endpoint `201 Created` is the more conventional choice.

3. Day 4 - Path operation configuration

Metadata parameters passed to the decorator configure how an endpoint appears in OpenAPI / `/docs`:

Parameter	Effect
<code>status_code</code>	sets the response status
<code>tags</code>	groups endpoints in the docs UI (your code: <code>["user"]</code> , <code>["items"]</code>)
<code>summary, description</code>	short and long endpoint descriptions
<code>docstring</code>	used as the description if none is given; Markdown is supported
<code>response_description</code>	describes the response specifically
<code>deprecated=True</code>	marks the endpoint as deprecated without removing it

```
@app.post(
    "/items/",
    tags=["items"],
    summary="Create an item",
    response_description="The created item",
)
async def create_item(item: Item) -> Item:
    """Create an item with all the info."""
    return item
```

4. Day 5 - Handling errors with HTTPException

Client errors use status codes in the 400 range. `HTTPException` is a Python exception with API-relevant data. You **raise** it rather than return it: raising stops execution immediately and FastAPI sends the error response.

```
@app.get("/items/{item_id}")
async def read_item(item_id: str):
    if item_id not in items:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item": items[item_id]}
```

`detail` can be any JSON-able value (string, dict, list). The client receives `{"detail": ...}`. `HTTPException` also accepts a `headers` dict for advanced cases:

```
raise HTTPException(
    status_code=404,
    detail="Item not found",
    headers={"X-Error": "Custom error"},
)
```

In your `update_item`: `raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f" {item_id} was not found in the system")` - verified to return 404 with that

detail.

5. Day 5 - Custom exceptions and handlers

`@app.exception_handler(SomeException)` registers a function that builds the response whenever that exception is raised anywhere in the app. Your code:

day4-5/main.py

```
class new_exception(Exception):
    def __init__(self, id):
        self.id=id

@app.exception_handler(new_exception)
async def new_exception_handler(request: Request, exc: new_exception):
    return JSONResponse(
        status_code=401,
        content=f"sorry but we can not find item_id {exc.id}"
    )
```

- 1 `read_item` raises `new_exception(item_id)` when the id is unknown.
- 2 FastAPI finds the handler registered for that class and calls it with the request and the exception.
- 3 The handler builds a `JSONResponse` itself – here using `exc.id` in the message.

In your code

401 means Unauthorized (not logged in). For an item that does not exist the correct code is **404 Not Found**. Verified: `GET /items/zzz` currently returns 401.

Overriding the default handlers

- `RequestValidationError` – override FastAPI's invalid-request handler to customize the error format. Its `.body` attribute contains the raw invalid body sent by the client.
- `StarletteHTTPException` – override the base `HTTPException` handler to also catch errors raised internally by Starlette.
- FastAPI's default handlers can be reused from `fastapi.exception_handlers` while adding side effects such as logging.

Common mistake

Avoid returning the raw string of `RequestValidationError`: it can expose internal file/line information.

6. Day 5 - Body updates: PUT vs PATCH

	PUT - full replace	PATCH - partial update
Client sends	the whole object	only the fields that change
Omitted fields	fall back to model defaults and can overwrite stored values	left untouched
Key tools	<code>jsonable_encoder(item)</code>	<code>model_dump(exclude_unset=True) + model_copy(update=...)</code>

Classic PUT (from the Day 5 summary)

```
@app.put("/items/{item_id}", response_model=Item)
async def update_item(item_id: str, item: Item):
    update_item_encoded = jsonable_encoder(item)
    items[item_id] = update_item_encoded
    return update_item_encoded
```

The exclude_unset technique

- 1 Load the stored data into a Pydantic model.
- 2 `item.model_dump(exclude_unset=True)` keeps only the fields the client actually sent.
- 3 `stored_model.model_copy(update=update_data)` creates an updated copy.
- 4 `jsonable_encoder(updated_item)` converts it before saving.

Your version in day4-5/main.py (comments shortened)

```
@app.put("/items/{item_id}")
async def update_item(item_id: str, item: Item) -> Item:
    if item_id not in items:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=...)
    org_item = items[item_id] # stored dict
    new_item = item.model_dump(exclude_unset=True) # only sent fields
    org_item_pydantic = Item(**org_item) # dict -> model
    final_item = org_item_pydantic.model_copy(update=new_item)
    items[item_id] = jsonable_encoder(final_item) # model -> JSON-able dict
    return final_item
```

Your route is a PUT, but it uses the PATCH technique – the Day 5 notes confirm the same `exclude_unset` + `model_copy` technique can also be used with PUT. Just remember that `@app.patch` is the conventional method for partial updates.

A detail worth noticing (verified)

PUT `/items/bar` with `{"name": "New"}` keeps bar's stored description, price and tax. But PUT `/items/foo` with `{"price": 99}` returns `"tax": 10.5` – stored foo had no tax, so `Item(**org_item)` filled in the model default, and that default is now saved. `exclude_unset` protects stored values; it cannot invent values that were never stored.

In your code

`items` is defined twice in the file. `read_item` looks the dict up when a request arrives, so it always sees the second dict: GET `/items/foo` returns `{"item": {"name": "Foo", "price": 50.2}}`.

7. Common mistakes

- Returning (instead of raising) an `HTTPException`.
- Using 401 for “not found” – use 404.
- Forgetting that `response_model` overrides the return type.
- Calling `model_dump()` without `exclude_unset=True` in a partial update – defaults overwrite stored values.
- Making update fields required: to allow omitting them, give them defaults (commonly `None`).
- Using one model for input and output when the input contains secrets.

Optional / later

For cleaner API design, use separate models such as `ItemCreate` and `ItemUpdate` (you will see this pattern again on Day 10 with `HeroCreate` / `HeroUpdate`).

8. Revision summary

- Return type or `response_model` → validate, filter, document the response; `response_model` wins.
- Separate input/output models (or inheritance) keep passwords out of responses.
- Set `status_code` with `status.HTTP_...` constants; 2xx success, 4xx client error, 5xx server error.
- `tags`, `summary`, `description`, `response_description`, `deprecated` shape `/docs`.
- `raise HTTPException(status_code, detail)`; custom exceptions need `@app.exception_handler`.
- PUT replaces; PATCH updates part. Partial update = `model_dump(exclude_unset=True)` + `model_copy(update=...) + jsonable_encoder`.

Source of the original summaries: fastapi.tiangolo.com/tutorial/response-model,
[/response-status-code](https://fastapi.tiangolo.com/tutorial/response-status-code/), [/path-operation-configuration](https://fastapi.tiangolo.com/tutorial/path-operation-configuration/), [/handling-errors](https://fastapi.tiangolo.com/tutorial/handling-errors/), [/body-updates](https://fastapi.tiangolo.com/tutorial/body-updates/).